RAG Project Instructions

Python Files, Notebook Walkthrough, Streamlit Deployment, and Final Submission Guide

Build a simple RAG project as Python files, plus one notebook that walks through the same pipeline with real output. Final submission includes a ZIP file, a GitHub repository link, a deployed Streamlit app link, and the notebook.

Goal

Build a simple retrieval-augmented generation project that follows the lab sequence from raw documents to a deployed Streamlit assistant. Alongside the production Python files, build one

notebook that walks through each pipeline stage with real printed output, so the reasoning behind each stage is visible, not just the final app.

```
documents -> preprocessing -> chunking -> vector representation -> vector store ->
context retrieval -> prompting -> Streamlit UI
```

Required File Structure

Your project must contain these files:

```
01_documents.py 02_preprocessing.py 03_chunking.py 04_vector_representation.py
05_create_chroma_store.py 06_retrieve_context.py 07_prompting.py streamlit_app.py
requirements.txt Simple_RAG_Lab.ipynb
```

The notebook is not a copy of the Python files. It imports and runs the same functions, then shows the actual output of each stage as a labeled table, with a short explanation of what the output shows.

What the Notebook Must Cover

1. Documents and preprocessing

• Load the document set from 01_documents.py and show the raw text. • Run preprocess_text from 02_preprocessing.py and show a before/after table for at least 3 documents.

ELEVOR IX AC A DEMY

2. Chunking

• Run build_chunks from 03_chunking.py and show the resulting chunk table: chunk_id, document_id, chunk_text.

• State the chunk_size and overlap used, and what changes if overlap is set to 0.

3. Vector representation and hybrid search

• Run hybrid_search from 04_vector_representation.py for at least 2 example questions. • Show the BM25 score, the embedding score, and the combined hybrid score per candidate, not just the

final ranking.

4. Vector store and retrieval

- Build the Chroma store with 05_create_chroma_store.py and confirm the chunk count matches the chunking stage.
- Run build_context from 06_retrieve_context.py and show which source was dropped for being OUTDATED, using the printing-price documents as the example.

5. Prompting and grounded generation

• Call answer_question from 07_prompting.py for the same example questions. • If OPENROUTER_API_KEY is not set, the notebook must say so plainly instead of failing silently — state this is the same defensive pattern used in the course labs. • Confirm the answer cites a source tag like [Source 1] and does not cite the OUTDATED document when a CURRENT one is available.

API Key Rules

• Do not write your real API key inside any Python file or notebook cell. •
Do not upload your real .env file.

- Use Streamlit TOML secrets for deployment.

Streamlit Secrets

In Streamlit Cloud:

1. Open your app.
2. Click Manage app.
3. Open Secrets.
4. Add the TOML values below.

```
OPENROUTER_API_KEY = "your_openrouter_key_here" OPENROUTER_MODEL = "openai/gpt-4o
mini"
```


Your app should read the key from Streamlit secrets when deployed:

```
try:
        if not rag.OPENROUTER_API_KEY:
        rag.OPENROUTER_API_KEY = st.secrets.get("OPENROUTER_API_KEY", "")
        rag.OPENROUTER_MODEL = st.secrets.get("OPENROUTER_MODEL",
        rag.OPENROUTER_MODEL)
```

```
except Exception: pass
```

Final Submission

Submit four items:

5. ZIP file of your project, including Simple_RAG_Lab.ipynb with outputs visible.
6. GitHub repository link.
7. Streamlit deployed app link.
8. The notebook link or file, if your platform hosts notebooks separately from the ZIP.

Final Checklist

• All required Python files exist.

• requirements.txt exists.

• Simple_RAG_Lab.ipynb exists, is fully run, and every output is visible. • Your real API key is not included in the ZIP file, GitHub repository, or notebook. • Streamlit secrets are configured in valid TOML format.

• The Streamlit app runs successfully.

• The answer uses retrieved context.

- The answer cites sources.